



Velocity Business OS

A plain-English walkthrough for the senior team

One system that runs a client from the first enquiry to the final report — sales, proposals, contracts, signatures, payments, delivery and reporting — where the **links between those stages are enforced by the software instead of remembered by people.**

STAGE	TESTS PASSING	DATABASE TABLES	API ENDPOINTS
Phase 1 done	176	80	59

Built on Next.js 15 + PostgreSQL (Supabase). Prepared September 2026.

0 What is in this document

Read sections 1–3 if you only have five minutes. Sections 4–6 are the screen-by-screen tour.

- | | | | |
|---|--|----|--|
| 1 | The problem we are solving — with a real example | 7 | The most important part: Won → Delivery |
| 2 | The client journey in one line | 8 | Who is allowed to do what (roles) |
| 3 | The dictionary — every term explained | 9 | AI and Automations — and their safety limits |
| 4 | Every tab in the app, one by one | 10 | The five design decisions that make this different |
| 5 | The Client 360 page and its 12 tabs | 11 | What is built, what is not, and what is next |
| 6 | The Pipeline in detail — stages and gates | 12 | Meeting notes: what to say, and likely questions |

The 30-second version

IF SOMEONE ASKS "WHAT IS IT?"

"Today a deal lives in a CRM, the proposal in Word, the contract in email, the project in a spreadsheet and the invoice in Tally. Nothing checks that they agree with each other. **Velocity OS is one system where they are the same record** — and where the system physically refuses to let delivery start before the agreement is signed, or a deal be marked Won before the client has accepted a proposal."

The numbers, if asked

80

TABLES

168

SECURITY POLICIES

98

DB SAFETY TRIGGERS

230

DATA RULES

127

PERMISSIONS

8

ROLES

59

API ENDPOINTS

176

AUTOMATED TESTS

All 176 tests run against a real PostgreSQL engine, not fake data — so the security rules being tested are the same ones running in production.

1 The problem we are solving

A services business does not lose money inside a tool. It loses money in the gaps *between* tools.

A story everyone in the room will recognise

WHAT HAPPENS TODAY

Sales closes a deal with Acme Ltd on Monday and shouts "we won it!" in the group. The delivery team, being helpful, starts work on Tuesday. The contract is still sitting unsigned in the client's legal inbox. Three weeks later the client walks away. **We have done three weeks of work with no signed agreement and no advance payment.** Nobody did anything wrong — no single person was responsible for checking.

WHAT HAPPENS IN VELOCITY OS

Sales marks the deal Won. The system automatically creates the Onboarding record and puts it in status **Blocked** with a visible checklist: *NDA signed, MSA signed, 30% advance received*. The project simply **cannot be created** until those are genuinely true — not ticked by a human, but proven by a signed document and a received payment. When the last one is satisfied, the onboarding turns **Ready** and the project is generated with its tasks and KPIs already filled in.

The four failures we set out to make impossible

THE FAILURE	HOW THE SYSTEM PREVENTS IT
Delivery starts before signing Because nobody checked.	The Legal Gate . The database itself refuses to move an onboarding out of "Blocked" while a required contract is unsigned or an advance is unpaid.
The project is planned from an old service description	The project plan is generated from the service that was actually sold on the accepted proposal — not typed by hand.
The agreement is generated from a stale proposal Sales edited the price after the client accepted.	The moment a client accepts, that proposal version is frozen . Later edits create a new version and never repoint the agreement.
Nobody can say who approved sending a contract, or when	An append-only audit log . Even a system administrator cannot edit or delete an entry — that permission has been removed at the database level.

Why not just buy the tools?

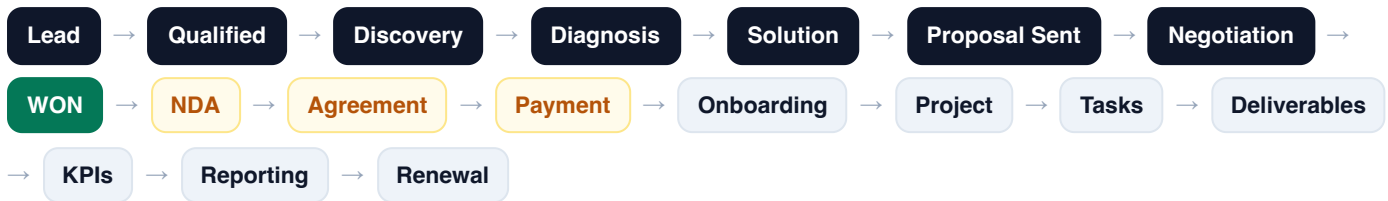
We could buy a CRM, a proposal tool, a contract tool and a project tool. Each is good. But none of them can enforce a rule that *spans* them — "no delivery without a signed MSA" is a sentence that crosses four products. That rule can only live in a system that owns all four records.

LINE FOR THE MEETING

"The value is not that everything is in one place. That is easy. The value is that the connections between stages are now rules the software enforces, instead of things we hope someone remembers."

2 The client journey in one line

Everything in the product sits somewhere on this line. If you understand this line, you understand the product.



Dark = the sales pipeline (the "Pipeline" tab). **Amber** = the Legal Gate — the three things that must be true before work begins. **Grey** = delivery.

Walk it once with a real example — "Acme Ltd wants a website"

STAGE	WHAT IT MEANS IN PLAIN WORDS	WHAT ACTUALLY HAPPENS FOR ACME
Lead	Someone showed interest. Nothing verified yet.	Acme fills the website form. The system creates the company, the contact and the opportunity in one go.
Qualified	We have proof this is a real deal.	Cannot move here until three things are filled in: the <i>business problem</i> , a <i>budget indication</i> , and a named <i>decision maker</i> . Missing any one, the system says exactly which.
Discovery	We are asking questions and gathering facts.	Meeting notes, current traffic numbers, competitor list, timelines — captured in the Discovery workspace.
Diagnosis	We state what is actually wrong, and where each statement came from.	"Bounce rate is 71%" is tagged as <i>client provided</i> . "Likely cause is page load time" is tagged as <i>our inference</i> . An untagged claim cannot be saved.
Solution	We pick services from the catalogue and price them.	Website Rebuild (₹8L) + 6 months SEO (₹1.2L/mo). All arithmetic is done by the database, never by JavaScript.
Proposal Sent	The priced document has gone to the client.	Cannot move here until the proposal has been internally approved . A salesperson cannot send an unapproved price.
Negotiation	They are pushing back on price, scope or timing.	Version 2 of the proposal is created. Version 1 is kept, untouched.
Won	The client has said yes, in writing, on a specific version.	Cannot move here until a proposal version is Accepted . That version is then frozen onto the deal forever.
Closed	Fully finished and archived. Nothing more will happen.	The end of the line. No further changes allowed.
Lost	We did not get it.	A reason is compulsory — price, timing, no budget, competitor, no decision, lost contact, not a fit, internal capacity, or other. This is what makes the loss report meaningful.
Dormant	Parked. Not lost, just not now.	The system remembers which stage it was parked from. Reactivating it puts it back exactly where it was.

THE SINGLE MOST IMPORTANT SENTENCE IN THIS DOCUMENT

A deal cannot be dragged from one stage to another just because someone wants to. Each arrow has a **guard** — a condition that must be true. And that guard lives in the **database**, not just in the screen. So even a developer running a direct database command, or a future feature that forgets to check, **still cannot skip it**.

3 The dictionary — every term explained

These are the words that will come up. Each has one meaning in this system, and only one.

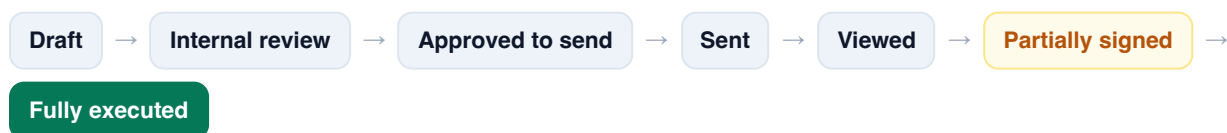
Sales terms

Lead a stage	A possible client we have not verified yet. There is no separate "leads list" — a lead is simply an opportunity sitting in the first stage, so nothing is lost or re-typed when it becomes real. Example: a form fill from the website.
Opportunity also: a deal	One potential piece of business with one client. It carries the value, the currency, the owner, the expected close date and the stage. One record travels the entire funnel from Lead to Won. Example: "Acme — Website Rebuild — ₹8,00,000 — Negotiation — owned by Priya."
Open Pipeline a number on the dashboard	The total money value of every deal that is still alive — i.e. in Lead, Qualified, Discovery, Diagnosis, Solution, Proposal Sent or Negotiation. It excludes Won, Lost, Closed and Dormant. It answers: "how much business are we currently chasing?" Example: 14 open deals worth ₹1.9 crore in total = open pipeline of ₹1.9 Cr.
Pipeline (the tab) a screen	A Kanban board — one vertical column per stage, one card per deal. You drag a card to move a deal forward. If the guard is not satisfied, the drag is refused and the reason is shown.
Won a stage	The client has formally accepted a specific version of our proposal. It is not a feeling or an email saying "go ahead" — the system requires an Accepted proposal version to exist before it will allow this. Winning the deal is what triggers the NDA and the onboarding checklist. Note: a Won deal can still become Lost — the client can walk before signing. It is not the point of no return; the signed agreement is.
Lost a stage	We did not get it. The system will not accept this without a reason code from a fixed list, so "why do we lose?" is answerable from data rather than from memory.
Dormant a stage	Parked deliberately — client says "revisit in March". Reversible: the system stores which stage it came from and restores it on reactivation. This keeps the pipeline number honest without deleting real opportunities.
Closed a stage	Finished and archived. This is the only truly final stage — nothing can move out of it.
Qualification a set of three facts	The evidence required before a deal is taken seriously: the business problem, a budget indication, and a named decision maker . All three, or the deal stays a Lead.
Discovery	The fact-gathering workspace. Notes, numbers, documents, meeting records.
Diagnosis with provenance	Our written assessment of the client's problem, where every claim must be labelled with where it came from — client provided, our inference, or an AI recommendation. An unlabelled claim cannot be saved. This is what stops a guess quietly becoming a fact in a proposal.
Solution	The chosen combination of services with quantities, discounts and totals. Every rupee is calculated by the database, so the proposal, the invoice and the report can never disagree by a rounding error.
Proposal & Proposal Version	The priced document sent to the client. Every revision is a numbered version . When the client accepts one, that version becomes immutable and everything downstream — the agreement, the project plan, the invoice — reads from it. Example: v1 ₹8L → client pushes back → v2 ₹7.2L accepted. The contract is generated from v2, and v2 can never be edited again.

Legal terms

NDA Non-Disclosure Agreement	The confidentiality agreement — "we will not share each other's private information". Usually the first thing signed after a deal is Won, because real delivery information starts flowing.
MSA Master Service Agreement	The main contract. It sets the overall terms of the relationship — payment terms, liability, IP ownership, termination. Signed once, then reused for all work with that client.
SOW Statement of Work	The specific piece of work under the MSA: what exactly will be delivered, by when, for how much. One MSA can have many SOWs. Analogy: the MSA is the rental agreement for the building; each SOW is a booking of a specific room.
Addendum / Amendment	A change to an existing contract. It is linked to its parent contract rather than replacing it, so the history stays readable.
Contract status the ten states	Draft Internal review Approved to send Sent Viewed Partially signed Fully executed Declined Expired Voided
Fully Executed the important one	Signed by everybody. This is the status that unlocks delivery. It is permanent and unchangeable — once a contract reaches it, the record is frozen. And it cannot be set by hand. See "Evidenced execution" below.
Evidenced execution how we prove signing	A contract only becomes Fully Executed after four things happen in order: (1) the e-signature provider sends us a verified notification, (2) we download the signed PDF ourselves, (3) we compute its digital fingerprint (SHA-256 hash), (4) we store it permanently. The database refuses "executed" if there is no executed document attached. Meaning: nobody can mark a contract signed just by clicking a button, and a fake notification from an attacker achieves nothing.
E-signature provider	The external service that collects signatures (we have built the Zoho Sign connector). There is also a manual mode where the whole legal workflow runs internally with uploaded signed PDFs — useful for clients who insist on wet signatures, and it is what our automated test uses.
Legal Gate the core safety rule	The checklist that stands between "Won" and "start work". It lists every contract that must be Fully Executed and every payment that must be received before delivery can begin. Enforced inside the database , so no screen, script or shortcut can bypass it.
Legal Override	The deliberate exception — a business decision to start work before the paperwork is complete. Only two roles can do it (Legal/Admin and Super Admin). It requires a written reason, it is recorded permanently, and it puts a banner on the client's page that never goes away . It is possible, but it is never quiet.

The contract lifecycle, in one line



Or it ends at **Declined**, **Expired** or **Voided**. **Approving** a contract and **sending** it are two different permissions, usually held by two different people.

Money terms

Payment Requirement	A rule about money that must be satisfied — "30% advance before kickoff", "40% on milestone 1". It is a requirement, not an invoice.
Invoice / Payment / Allocation	The invoice is the bill. The payment is money that arrived. The allocation is which payment settles which invoice — because clients often pay two invoices with one transfer.
Threshold evaluation "advance received"	Whether the advance is satisfied is calculated live from actual payments , never stored as a yes/no tick. So it cannot drift from reality, and it cannot be ticked by someone in a hurry.
Base currency & FX	Each deal keeps its own currency. The exchange rate is captured at the moment of the transaction and stored with it, so last year's reports do not change when today's rate moves.
Margin & Cost	Profit information. It is a separately controlled permission — Sales cannot see it. It is removed on the server before the data leaves, not merely hidden in the screen, so it is not visible even to someone inspecting the network traffic.

Delivery terms

Onboarding	The bridge between a Won deal and a live project. It holds the checklist of required documents and payments. Its statuses are Blocked → Ready → In progress → Complete (or Cancelled). It starts at Blocked, always.
Provisioning "generating the project"	The automatic creation of the project from what was sold — its workstreams, its tasks, its deliverables, its KPIs and its reporting rhythm, all copied from the service catalogue entries on the accepted proposal. Example: choosing "SEO Retainer" automatically creates 14 recurring tasks, 3 KPIs and a monthly report cadence. Nobody types them.
Project	The container for all delivery work for one engagement.
Workstream	A named chunk of a project. Example: a website project has Design, Content, Build and Launch workstreams.
Task	One unit of work with an owner and a due date. Tasks can depend on other tasks. Statuses: To do, In progress, Done, Skipped, Blocked.
Deliverable	The actual thing handed to the client — the report, the design file, the live site. Distinct from a task: a task is effort, a deliverable is the output.
KPI Key Performance Indicator	The measurable promise. Each KPI has a target, dated measurements, and a status the system derives (on track / at risk / off track) rather than someone declaring it. Example: "Organic traffic +40% in 6 months." Measured monthly; the status is computed from the trend.
Cadence	The agreed rhythm of reporting and reviews — weekly status, monthly report, quarterly business review.
SLA clock	Time measured in working hours against a business calendar, so a ticket raised on Friday evening is not "overdue by 60 hours" on Monday morning.
Renewal	The end of a retainer period, and the decision to continue. The data model and the background sweep exist; the forecasting screen is a later phase.

System terms (only if a technical question comes up)

Multi-tenant & RLS Row Level Security	Several organisations share one database, but PostgreSQL itself filters every single query by organisation. Not a filter in our code that could be forgotten — a rule the database applies to every read and write. 168 such policies are in place, and a dedicated test suite tries to break out of one tenant into another and fails.
Audit log	A permanent record of every consequential action: who, what, when, from where. It is append-only — the ability to edit or delete an entry has been revoked at the database level, for everyone.
Activity timeline	The human-readable story on a client or deal page — emails, stage moves, notes, uploads, signatures — in one chronological list.
Transition channel	The single controlled doorway through which a stage or status may change. A direct database command such as <code>UPDATE opportunities SET stage='won'</code> is rejected . This is what makes the guards trustworthy: if state could move without them, they would be decoration.
Event / Outbox	When something meaningful happens ("opportunity.won"), a durable event is written in the same transaction as the change itself. So the fact and the announcement of the fact can never disagree — and if the change is rolled back, no event survives.
Worker	A separate background process that picks up those events and does the slow jobs — running automations, downloading executed contracts, scheduled sweeps. The app works without it; the automations do not.
Idempotency	Doing the same thing twice has the same effect as doing it once. So a double-clicked button or a retried network call does not create two invoices.
Signed URL	How files are served. No file is public. Each download gets a private link that expires in 15 minutes , is issued only after a permission check, and is written to an access log.
Immutable / append-only	"Can be added to, never changed or removed." Applies to the audit log, executed contracts, accepted proposal versions and document versions.

If you remember only four of those

ROW LEVEL SECURITY

Our data separation is enforced by PostgreSQL itself, not by our code. It cannot be forgotten by a future feature.

TRANSITION CHANNEL

Stages cannot move except through the guarded doorway. This is what makes every other rule trustworthy.

APPEND-ONLY AUDIT LOG

Who did what, when. Nobody can edit or delete it — including administrators.

SIGNED URLS

No file is ever public. Every download is permission-checked, expires in 15 minutes, and is logged.

4 Every tab in the app, one by one

Four groups in the left sidebar. **The menu adapts to the person** — a salesperson does not see Finance at all. (The server enforces that regardless; hiding the link is only courtesy.)

1 — THE CORE

Dashboard · Pipeline ·
Clients · Services

2 — DELIVERY

Projects · Tasks ·
Documents

3 — COMMERCIAL

Legal · Finance · Reports

4 — INTELLIGENCE

AI · Automations · Settings

1. Dashboard /dashboard

IN ONE LINE "What should I worry about today?"

WHAT YOU SEE The pipeline funnel (how many deals and how much money at each stage), delivery health, overdue tasks, contracts awaiting signature, and — most importantly — **blocked onboardings**: deals we have won but cannot start.

THE CLEVER BIT The top banner picks the single most urgent thing by *actual urgency*, not by position on the page.

EXAMPLE "3 onboardings blocked · 2 contracts awaiting signature · 5 of your tasks overdue"

2. Pipeline /pipeline

IN ONE LINE Every open deal, arranged by stage. This is the sales team's home screen.

WHAT YOU SEE A Kanban board — one column per stage, one card per deal showing client, value and owner. Drag a card right to advance the deal.

THE CLEVER BIT The drag is **a real transition, with guards**. Drag a deal into "Won" with no accepted proposal and it snaps back with: *"An accepted proposal is required before an opportunity can be won."*

CLICK A CARD Opens the deal detail page with a **qualification checklist** that mirrors the guard exactly — so you can see precisely what is missing before you try to move it.

EXAMPLE Priya drags "Acme — Website Rebuild" from Negotiation to Won. Blocked. She sends v2 for acceptance, the client accepts, she drags again — it moves, and an NDA task appears automatically.

3. Clients /clients

IN ONE LINE Every company we sell to and deliver for.

WHAT YOU SEE A searchable, sortable list. Clicking one opens **Client 360** — the single most valuable screen in the product, covered in section 5.

HANDLES Parent companies and group structures — so a client with five subsidiaries rolls up into one relationship view rather than five disconnected accounts.

4. Services /services

IN ONE LINE The catalogue of what we sell — and this is where the automation actually comes from.

WHAT YOU SEE A card per service with its price, its delivery duration, its default tasks, its default KPIs, and **the documents that must be signed before it can be delivered**.

WHY IT MATTERS MOST Defining a service once here means every future sale of it gets the correct project plan, the correct KPIs and the correct legal requirements automatically. **This is the leverage point of the whole system.**

EXAMPLE "SEO Retainer — ₹1,20,000/month — 6 months — requires MSA + SOW — 14 default tasks — 3 KPIs." Sell it, and all of that appears without anyone typing it.

5. Projects /projects

IN ONE LINE	Delivery in flight.
WHAT YOU SEE	Every live project, its client, its status, its progress and its health. Inside a project: workstreams, tasks, deliverables and KPIs.
THE CLEVER BIT	A project cannot be created by hand out of nowhere — it is provisioned from a Won deal that has passed the Legal Gate . So every project in this list is, by construction, a project we are contractually entitled to be doing.

6. Tasks /tasks

IN ONE LINE	"What needs doing, and by when."
WHAT YOU SEE	Your tasks by default, with a switch to your team's or the whole organisation's — depending on your permission. Overdue items are marked.
NOTE	Due dates use a business calendar , so weekends and holidays do not create fake overdue items.

7. Documents /documents

IN ONE LINE	Every file — uploaded or generated — versioned and tamper-evident.
THE RULES	Nothing is ever overwritten. Uploading again creates version 2 and keeps version 1. Every version gets a SHA-256 fingerprint at upload. Files are never public — each download is a permission-checked link that expires in 15 minutes and is logged.
THE CLEVER BIT	If a file's fingerprint ever stops matching its stored value, the file is automatically quarantined and a critical alert is raised. That is tamper detection, not just storage.
EXAMPLE	A signed MSA is downloaded from the e-signature provider, hashed, sealed, and can never be replaced — only superseded by a linked amendment.

8. Legal /legal

IN ONE LINE	NDAs, agreements, and everything waiting for a signature.
WHAT YOU SEE	Every contract with its type (NDA / MSA / SOW / Amendment) and status. Clicking one shows the full history and the actions <i>you personally</i> are allowed to take.
THE SEPARATION THAT MATTERS	Drafting, approving and sending a contract are three different permissions. And the authority to send is independent of who owns the deal — a salesperson who owns the client, owns the contract and wrote it <i>still cannot send it</i> . Only Legal/Admin and Super Admin can.
GENERATION	Contracts are rendered from templates with variables filled from real data. If a variable has no value, generation fails loudly rather than producing a contract containing "[CLIENT NAME]".

9. Finance /finance

IN ONE LINE	Money in, money owed, and what delivery is waiting on.
WHAT YOU SEE	Revenue for the period, outstanding amounts, the payment list, and which payment requirements are currently blocking an onboarding.
THE ENGINEERING PROMISE	Money is never a decimal-point number in code. It is exact numeric in the database, exact strings over the network, and every total is computed by PostgreSQL. There is exactly one answer to "what is the total", computed in one place.
ACCESS	Requires the finance permission. Sales does not have it.

10. Reports /reports

IN ONE LINE	The numbers, for a date range you choose, in the organisation's base currency.
WHAT YOU SEE	Won value and count, conversion rates, the pipeline funnel, loss reasons, delivery health, KPI attainment.
HONEST BY DESIGN	If an exchange rate for a currency has not been loaded, the report shows nothing for it rather than quietly assuming 1:1. A blank is recoverable; a wrong number that looks right is not.

11. AI /ai

IN ONE LINE	An assistant that can read your business data and answer questions — and can <i>propose</i> work, but never commit it.
TWO HALVES	Assistant — ask a question in plain English. Pending Actions — a queue of things the AI has drafted, waiting for a human to approve or reject.
THE SAFETY DESIGN	The AI has eight fixed, pre-written read tools and nothing else. It <i>cannot write its own database queries</i> . It can search clients, summarise the pipeline, list at-risk clients, list contracts by status, open a client 360, list overdue tasks, list renewals due, and get a revenue summary. That is the entire list.
APPROVAL IS ENFORCED	An AI action cannot be executed unless it has been approved — and that is enforced by a database trigger , not by the screen. There is also an organisation-level kill switch : turn AI off and no data from this organisation is sent to a model under any circumstance.
EXAMPLE	"Which clients are at risk this month?" → a list with reasons. "Draft the diagnosis for Acme" → a draft appears in Pending Actions with every claim labelled by source, for a human to approve.

12. Automations /automations

IN ONE LINE	WHEN something happens, IF a condition holds, THEN do these things.
WHAT YOU SEE	The list of rules, whether each is active, and a log of the last 25 runs with their outcome and duration.
THE SINGLE MOST IMPORTANT LIMITATION	The list of things an automation may do is closed and short : draft an email, generate a contract, notify someone, create a task, assign an owner, add a tag, set an allow-listed field, create a payment requirement, start an onboarding, request an AI action, add an activity. There is no "send email" action. There is no "send contract" action. They were not left out; they deliberately do not exist. A machine may prepare the work; a person with the right permission decides whether it leaves the building.
LOOP PROTECTION	Automations that trigger automations are limited by chain depth and a per-record cooldown, both recorded — so a rule cannot run away with itself.
EXAMPLE	WHEN an opportunity is Won · IF the value is over ₹5,00,000 · THEN generate the NDA, create the payment requirement, notify Legal, and start the onboarding. Four steps that used to be four people remembering.

13. Settings /settings

IN ONE LINE	Your organisation, its people, their roles, and your own access.
WHAT YOU SEE	Members with their roles and last-seen time, the organisation's base currency and preferences, the AI on/off switch, and the exact list of permissions you personally hold.

ALSO WORTH DEMONSTRATING LIVE

⌘K command bar — press Command-K anywhere and type. It searches clients, contacts, opportunities, projects, tasks and contracts at once, and jumps you straight there. Plus a **notification bell**, an **organisation switcher**, **dark mode**, and full keyboard navigation.

5 The Client 360 page — and its 12 tabs

Open any client and you get one page that answers every question anyone will ever ask about that relationship. This is the screen to demonstrate if you only demonstrate one.

TAB	WHAT IT ANSWERS	NOTES
Overview	The headline numbers: open pipeline, won value, active projects, outstanding money, contract count.	Built
Contacts	Every person we deal with here, and their role.	Built
Pipeline	Every deal with this client, open and closed.	Built
Services	What they have bought from us.	Panel pending data exists via API
Projects	Delivery in flight for them.	Built
Tasks	Open work for this client.	Panel pending data exists via API
Documents	Every file, versioned and hashed.	Built
Legal	Every contract and its status.	Built
Emails	Correspondence history.	Panel pending outbound only in Phase 1
Reports	KPI attainment for this client.	Panel pending data exists via API
Billing	Invoices, payments, outstanding.	Built — finance permission only
Activity	The full chronological story of the relationship.	Built

THE PERMANENT LEGAL BANNER

If anyone has ever used a Legal Override for this client — started work before the paperwork was complete — a banner appears at the top of this page and **never goes away**. Not for the person who approved it, not after the contract is eventually signed, not for anyone.

Why: the exception must remain visible to whoever picks up the account next year. An override that fades from view is an override nobody learns from.

LINE FOR THE MEETING

"One page. Twelve tabs. Everything anyone has ever asked me about a client in a review meeting is on this screen — and every number on it is live, not typed into a slide the night before."

What is honestly not finished here

Four of the twelve panels — Services, Tasks, Emails and Reports — currently show an explanatory placeholder. **The underlying data exists and is reachable through the API today**; only those four visual panels are unbuilt. It is presentation work, not foundation work. Worth stating plainly rather than being asked.

6 The Pipeline in detail — stages and gates

This is the table to put on screen if someone asks "so what actually stops a mistake?"

MOVING INTO...	...REQUIRES	WHAT THE USER IS TOLD IF IT FAILS
Qualified and every stage after it	Business problem (real text, not one word) + budget indication + named decision maker.	"Cannot move to Qualified until qualification details are complete. Missing: budget_indication, decision_maker."
Proposal Sent	All of the above, plus an internally approved proposal version.	"An internally approved proposal version is required before sending."
Won	All of the above, plus an Accepted proposal version. That version is then frozen onto the deal.	"An accepted proposal is required before an opportunity can be won."
Lost	A reason code from the fixed list.	"A lost opportunity must record why it was lost."
Dormant	A reason, and optionally a wake-up date. The current stage is remembered.	—

Which moves are even legal

Not every stage can reach every other stage. The permitted moves are defined explicitly:

FROM	CAN MOVE TO
Lead	Qualified · Lost · Dormant
Qualified	Discovery · Solution · Lost · Dormant
Discovery	Diagnosis · Solution · <i>back to Qualified</i> · Lost · Dormant
Diagnosis	Solution · <i>back to Discovery</i> · Lost · Dormant
Solution	Proposal Sent · <i>back to Diagnosis</i> · Lost · Dormant
Proposal Sent	Negotiation · Won · <i>back to Solution</i> · Lost · Dormant
Negotiation	Won · <i>back to Proposal Sent</i> · Lost · Dormant
Won	Closed · Lost — because a client can still walk away before signing
Lost	Qualified · Dormant — deals do come back
Dormant	Back to whichever stage it was parked from
Closed	Nothing. This is the only truly final stage.

THE POINT WORTH LANDING

Every one of these rules exists in **two places at once**: in the application, where it produces a clear message a person can act on; and in the database, where it makes the rule *actually true* — for a script, for a support engineer working directly on the database, and for a feature we write two years from now that forgets to ask. **An automated test checks that the two never disagree.**

7 The most important part: Won → Delivery

This is the workflow that justifies the whole project, and it is fully built and proven end to end by an automated test.

The seven steps, in order

#	STEP	WHAT THE SYSTEM DOES — AND REFUSES TO DO
1	Deal is marked Won	Only allowed if a proposal version is Accepted. That version is frozen onto the deal permanently.
2	Onboarding created , status Blocked	Its checklist is built <i>from the services actually on the accepted proposal</i> — so the required paperwork matches what was really sold.
3	NDA generated and sent	Rendered from a template with real values. A missing variable stops generation. Only Legal/Admin or Super Admin may send it.
4	Signature collected	Via the e-signature provider, or manually with an uploaded signed PDF.
5	Contract becomes Fully Executed	Only after: a <i>verified</i> webhook arrives → we download the signed file ourselves → we hash it → we store it immutably. Then, and only then, the status moves. It is permanent from that point.
6	MSA and advance payment	Same evidenced process for the MSA. "Advance received" is <i>calculated from actual payments</i> , never ticked by a person.
7	Onboarding turns Ready → project provisioned	Workstreams, tasks, deliverables, KPIs and reporting cadence are all generated from the service catalogue. Delivery begins.

WHAT IS GENUINELY IMPOSSIBLE

- Creating a project for a Won deal whose contracts are unsigned — **the database refuses**.
- Marking a contract as signed by hand, without a signed document to prove it.
- An unverified webhook marking a contract executed. Without a configured secret, webhooks are **stored and rejected, never processed**.
- A salesperson sending a contract, even one for their own deal that they drafted themselves.
- Editing an accepted proposal version to change what the agreement says.

AND WHAT IS DELIBERATELY STILL POSSIBLE

Business sometimes has to move before legal does. So a **Legal Override** exists. Only Legal/Admin and Super Admin can use it, a written reason is compulsory, it is recorded permanently in the audit log, and it leaves a banner on the client's page forever. **The system does not stop the business — it stops the business from doing it quietly.**

LINE FOR THE MEETING

"This whole seven-step chain runs as an automated test against a real PostgreSQL database on every single change we make. If someone breaks it, we find out in seconds — not when a client asks why we started without a contract."

8 Who is allowed to do what

Permissions are written as `resource : action : scope` — for example `opportunity:read:team` means "can read opportunities belonging to my team".

The three scopes

SCOPE	MEANS
own	Only records I personally own.
team	Records owned by me or anyone in my team.
org	Everything in the organisation.

Scopes exist so we do not need three separate roles for "sees own deals", "sees team deals" and "sees all deals". One role, one scope value. **127 permissions across 8 roles.**

The eight roles

ROLE	CAN	NOTABLY CANNOT
Super Admin	Everything.	—
Management	See everything including margin and cost.	Cannot override the Legal Gate.
Legal / Admin	Send contracts; override the Legal Gate.	The only role besides Super Admin with these two authorities.
Finance	Own all money — invoices, payments, revenue.	No legal authority.
Sales	Work their team's pipeline.	Cannot send contracts. Cannot see margin or cost.
Project Manager	Own delivery across the organisation.	Not commercial or legal authority.
Delivery	Their own tasks.	Deliberately narrow.
Client	Reserved for the future client portal.	Not active in Phase 1.

The three separations that were designed on purpose

1. **Sending a contract is not tied to owning the deal.** A salesperson who owns the client, owns the contract and drafted it still cannot send it. There is deliberately no "send my own contracts" permission — creating one would reintroduce exactly the coupling this removes.
2. **Approving a contract and sending it are different permissions.** In most organisations they are different people, and the system should not assume otherwise.
3. **Margin, cost and internal notes are separately controlled.** And they are *removed on the server* before the data is sent — not hidden by the screen. Someone inspecting the network traffic sees nothing.

How this is actually enforced

Not by `if (user.role === 'admin')` scattered through the code. By **168 policies inside PostgreSQL** that filter every read and write. A test suite deliberately attempts to read another organisation's data — by direct read, by write, by join, and by forging the organisation context — and every attempt fails.

LINE FOR THE MEETING

"The security is not in our code. It is in the database, underneath our code. So it still holds for a feature we write next year and for a query someone writes carelessly."

9 AI and Automations — and their limits

Expect this question: *"Is the AI going to email a client something wrong?"* The answer is architectural, not a promise.

The governing principle

The AI proposes. A person approves. Only then does anything happen. Automations may *draft* an email — there is no action that *sends* one. There is no action that sends a contract either. Those actions do not exist in the code.

What the AI can read — the complete list

Eight fixed, pre-written tools. The AI **cannot write its own database queries**; there is no text-to-SQL anywhere in the system.

<code>search_clients</code> — find companies	<code>get_client_360</code> — one client's full picture
<code>get_pipeline_summary</code> — deals by stage	<code>list_overdue_tasks</code> — what is late
<code>list_at_risk_clients</code> — accounts in trouble	<code>list_renewals_due</code> — what is coming up
<code>list_contracts_by_status</code> — legal position	<code>get_revenue_summary</code> — money, permission-checked

Every tool runs **as the person asking**, under the same security policies. The AI cannot see anything the user could not see by clicking around themselves.

What the AI can draft

A diagnosis · a proposal section · an email · suggested contract variables · a discovery summary · meeting notes · suggested tasks · suggested KPIs · a client report · an activity classification. **Every one of these lands in the Pending Actions queue for a human.**

The five safety layers

LAYER	WHAT IT DOES
Provenance	Every AI claim must be labelled: client-provided, AI inference, or AI recommendation. An inference must also carry a confidence value. Unlabelled output is rejected.
Approval trigger	The database itself refuses to execute an AI action that has not been approved. Not a screen check — a database rule.
Fixed tools	Eight read-only tools, no query generation.
Injection defence	Text that arrives from outside — a client's email, an uploaded document — is treated as data, never as instructions to the model. This is tested.
Kill switch	One organisation-level setting. Turn it off and no data from this organisation reaches a model at all .

Automations — the eleven permitted actions

`draft_email` `generate_contract` `notify` `create_task` `assign_owner` `add_tag` `set_field` `create_payment_requirement`
`start_onboarding` `request_ai_action` `add_activity`

Note what is *not* on that list: sending anything, moving a deal's stage, marking a contract executed, recording a payment. And `set_field` can only touch a short allow-list of harmless attributes — never a lifecycle field.

LINE FOR THE MEETING

"We automated the work, not the commitment. The machine can prepare the NDA, create the tasks and notify Legal. A human with the right authority still decides whether it leaves the building."

10 The five decisions that make this different

If a senior technical person is in the room, these are the five things to say.

1 — EVERY RULE IS ENFORCED TWICE, ON PURPOSE

Each important rule exists both in the application and in the database. They are not redundant; they do different jobs.

The application check produces a message a human can act on ("missing: budget_indication, decision_maker").

The database check makes the rule true — for a script, for a direct database session, for a future feature that forgets to ask.

The cost, stated honestly: the same rule is written twice, in two languages. A test asserts they never drift apart. We pay that price deliberately.

2 — CHANGING A STAGE IS A CHANNEL, NOT A CONVENTION

"Never change a status directly" is the kind of rule that holds until the week someone is in a hurry. So a direct `UPDATE ... SET stage='won'` is **rejected by the database**. Lifecycle fields can only move inside a transaction the transition service has explicitly marked as safe. Without this, every guard in the system would be decoration.

3 — MONEY IS NEVER A FLOATING-POINT NUMBER

Exact numeric in the database, exact strings over the network, exact decimal objects in between. **Every total is computed by PostgreSQL** — so there is exactly one answer to "what is the total", produced once. The proposal, the invoice and the report cannot disagree.

4 — EXECUTION IS EVIDENCED, NOT ASSERTED

A contract does not become "signed" because someone clicked a button or because a message arrived claiming so. It becomes signed after a *verified* notification, the signed file downloaded *by us*, hashed *by us*, and stored immutably.

The database refuses "executed" with no executed document. The status is a consequence of the evidence, not a substitute for it.

5 — HUMANS HOLD THE CONSEQUENTIAL DECISIONS

The AI proposes an action a person approves. Automations may draft an email; there is no send-email action, and no send-contract action. This is not a configuration setting someone can turn on — **those capabilities were never written.**

Two more worth knowing if asked

DECISION	WHY
We talk to PostgreSQL directly, not through Supabase's auto-API	Winning a deal writes four tables — the deal, the history ledger, the event, the audit entry. Those are one fact about the world. If any could succeed without the others, the system would lie. Only a real transaction can guarantee that. The database security rules still apply to every query.
Events are stored in the database, in the same transaction	A deal cannot be marked Won without its "opportunity.won" event also existing, and no event survives a rollback. That is not achievable by publishing to a message queue after the fact.

A CREDIBILITY POINT, IF YOU WANT ONE

The test suite found **twelve genuine bugs** that only a test against a real PostgreSQL engine could have caught. One example worth telling: a table had permissions to read and create rows but not to update them — so writing back the e-signature provider's reference *silently affected zero rows and reported success*. Every executed contract would have been unretrievable, and nothing would have looked wrong. That class of bug does not surface in a demo. It surfaces six months later, in front of a client.

Built and verified — Phase 1

SALES

Companies, contacts, group structures · one opportunity record for the whole funnel · lead capture in a single transaction · the state machine with guards · an append-only history ledger · lost reasons · reversible dormancy · Discovery · Diagnosis with provenance · the service catalogue · the solution builder · proposals with versioning and a frozen accepted version

LEGAL

Templates and a deterministic renderer · the full contract lifecycle · approve and send as separate authorities · the Zoho Sign connector and a manual mode · verified-webhook processing with retries · evidenced execution · amendments

FINANCE

Payment requirements, invoices, payments, allocations · derived threshold evaluation · multi-currency with rates captured at transaction time · structured so an external accounting system can take over later

DELIVERY

Onboarding with materialised requirements · the Legal Gate enforced in the database · the audited override · project provisioning · projects, workstreams, tasks, dependencies, comments, deliverables · the KPI framework · a business calendar and SLA clock

INTELLIGENCE & PLATFORM

The AI approval framework · eight read tools · injection defences and a kill switch · the automation engine with loop protection · the audit log · the event outbox · the job queue and worker · notifications · rate limiting · idempotency

INTERFACE

Sidebar shell · ⌘K command bar · permission-aware navigation · dashboard · Client 360 · the pipeline Kanban · legal screens · projects, tasks, services, documents, finance, reports, AI, automations, settings · dark mode, responsive layouts, keyboard access

Not built yet — planned, and the foundation supports each without rework

ITEM	CURRENT POSITION
Client portal	The data contract for it already exists. It is deliberately a separate projection — hiding fields in the screen is not a security boundary.
Two-way Gmail / Outlook	Phase 1 is outbound only, by design.
Renewal engine and forecasting	The data model and the background sweep exist; the screens do not.
Drag-and-drop automation builder	Automations are already stored as data, so this is a UI on top of a working engine.
WhatsApp and Slack	Not started.
Delivery profitability reporting	Not started.
Browser-level tests	176 tests cover logic, permissions, state machines and the full workflow. Browser tests would additionally cover rendering and clicking.
Four Client 360 panels	Services, Tasks, Emails, Reports. Data exists via API; the panels are unbuilt.

Known limitations — state these before you are asked

- **The Zoho Sign connector has not been run against the live Zoho API.** It is written to their documented API and its webhook verification is tested, but no real calls have been made. Manual mode covers the entire workflow and is what our end-to-end test uses. *Needs a real Zoho account to close out.*
- **File storage is simulated in tests.** Hashing, versioning and immutability run for real; the storage client itself is substituted.
- **Exchange rates must be loaded** before multi-currency reports show totals. The system returns nothing rather than assuming 1:1 — deliberate, and documented.
- **Node 20 is recommended** before deployment. Everything works on 18, but one library warns.

The engine is built, tested and correct. The remaining work is surface area — more screens, more connectors, and one live integration to verify against a real provider account.

A ten-minute running order

TIME	SHOW — AND SAY
0–1	Nothing. The Acme story from section 1. Everyone recognises it.
1–2	The journey line. "Every screen sits somewhere on this line."
2–4	Pipeline board. Drag a deal into Won with no accepted proposal. Let it fail on screen. That refusal is the entire product in one gesture.
4–6	Client 360. "One page, twelve tabs, every number live."
6–8	Onboarding, Blocked. The Legal Gate checklist. "The project cannot be created until these are genuinely true."
8–9	Automations. "Eleven permitted actions. Sending is not one of them."
9–10	Status slide. Section 11 — built, not built, and the four known limitations.

Questions you should expect

Q. Why build this instead of buying Salesforce / HubSpot / Zoho One?

Each is excellent at its own stage. None can enforce a rule that crosses stages — "no delivery without a signed MSA" spans four different products, so it can only live in a system that owns all four records. We also own our data model, so a new service type or gate is a configuration change, not a vendor request.

Q. What if the system blocks something the business genuinely needs to do?

The Legal Override exists for exactly that. Two roles can use it, a written reason is compulsory, it is permanently audited, and it leaves a banner on the client's page forever. It does not stop the business — it stops the business doing it quietly.

Q. Is client data safe? We hold contracts and financials.

168 policies inside PostgreSQL filter every read and write by organisation — not a filter in our code that could be forgotten. Files are never public; each download is a permission-checked link expiring in 15 minutes, and every access is logged. The audit log cannot be edited or deleted by anyone, and a test suite performs every attack we could think of.

Q. Will the AI do something embarrassing?

It cannot send anything — there is no send action in the code. It reads through eight fixed tools, cannot write its own queries, and runs under the asking user's permissions. Everything it produces waits in a queue for a human. And one switch turns it off entirely.

Q. How do we know it works? Have you tested it?

176 automated tests against a real PostgreSQL engine rather than mock data — so the policies being tested are the ones running in production. The full Lead-to-Onboarded workflow is one of those tests, run on every change. They found twelve real bugs that would otherwise have shipped.

Q. How long until we can use it for real clients?

The engine is done. Before a live pilot: verify the e-signature connector against a real provider account, load the service catalogue with our real services and their contract requirements, and load exchange rates. Suggest a pilot with one team on a few live deals.

Q. What is the biggest risk?

Adoption, not technology. A system that enforces rules is more work at the start than a spreadsheet that enforces nothing. That is the point, and also the friction. The mitigation is the Services catalogue: configure it once and the system gives back far more than it asks, because the project plan, the KPIs and the legal requirements stop being anyone's job.

The three sentences to close with

1. **We do not lose money inside our tools. We lose it in the gaps between them.**
2. **This system turns those gaps into rules the software enforces, instead of things we hope someone remembers.**
3. **The engine is built and proven. What is left is surface area.**